ELEC 475: Lab 2 Post-Lab Report

From LeNet-5 to AlexNet: Normalization Study, Data Augmentation and Dual-Dataset Benchmark

Course	ELEC 475, Computer Vision with Deep Learning (Fall 2026), Queen's University
Lab	Lab 2, Post-Lab (Instructor: Dr. Michael Greenspan)
Student	Ray Goldberg, student number 204025837
Partner	Jack Dorland (20405302)

All experiments were run on: **Tesla T4**. Mean epoch time is the wall-clock time of one training epoch plus one validation pass, averaged over all epochs of the run (train.py Timer). Validation metrics use the official CIFAR-10 test split and the official Tiny ImageNet validation split. Optimizer: Adam (lr 1e-3, weight decay 1e-4), batch size 128, cross-entropy loss, global seed 42 unless stated. Unless noted, validation accuracy and Macro F1 are taken at the epoch of best validation accuracy (the epoch whose weights are saved as the checkpoint).

1. Architectural Specifications

Spatial size follows N_out = floor((N_in - K + 2P) / S) + 1. Receptive field (RF) is accumulated layer by layer as RF_l = RF_(l-1) + (K_l - 1) * J_(l-1), with jump J_l = J_(l-1) * S_l, starting from RF = 1 and J = 1. Output shapes are (C x H x W) for a 3x64x64 input and 10 classes. Parameter counts are exact (weights plus biases, measured from the instantiated model).

Base LeNet-5: 338,486 parameters, first dense layer input = 2,704

Two 5x5 convolutions with Tanh and 2x2 average pooling (no padding). 64 -> 60 -> 30 -> 26 -> 13, so the flatten size is 16 x 13 x 13 = 2,704.

Layer	Output (CxHxW or F)	Params	RF (px)
Conv2d 3->6, k5, s1, p0	6x60x60	456	5
AvgPool2d k2, s2	6x30x30	0	6
Conv2d 6->16, k5, s1, p0	16x26x26	2,416	14
AvgPool2d k2, s2	16x13x13	0	16
Linear 2704->120	120	324,600	
Linear 120->84	84	10,164	
Linear 84->10	10	850	

Modernized LeNet: 446,006 parameters, first dense layer input = 3,600

Same two convolutions with padding 2, ReLU and overlapping 3x3 stride-2 max pooling. 64 -> 64 -> 31 -> 31 -> 15, so the flatten size is 16 x 15 x 15 = 3,600.

Layer	Output (CxHxW or F)	Params	RF (px)
Conv2d 3->6, k5, s1, p2	6x64x64	456	5
MaxPool2d k3, s2	6x31x31	0	7
Conv2d 6->16, k5, s1, p2	16x31x31	2,416	15
MaxPool2d k3, s2	16x15x15	0	19
Linear 3600->120	120	432,120	
Linear 120->84	84	10,164	
Linear 84->10	10	850	

AlexNet-64 (BatchNorm variant shown): 6,611,402 parameters, first dense layer input = 2,304

Five convolution stages (64, 192, 384, 384, 256 filters), overlapping max pooling after stages 1, 2 and 5, flatten 256 x 3 x 3 = 2,304, then Dropout(0.5)-FC 1024-ReLU-Dropout(0.5)-FC 1024-ReLU-FC classes. The 'none' variant removes the normalization layers; the 'lrn' variant removes BatchNorm and adds LocalResponseNorm(5, 1e-4, 0.75, 2) after ReLU in stages 1 and 2 (Conv -> ReLU -> LRN -> MaxPool, as in the 2012 paper). Neither changes the parameter count except BatchNorm (+2 per channel).

Layer	Output (CxHxW or F)	Params	RF (px)
Conv2d 3->64, k5, s2, p2	64x32x32	4,864	5
BatchNorm2d(64)	64x32x32	128	
MaxPool2d k3, s2	64x15x15	0	9
Conv2d 64->192, k5, s1, p2	192x15x15	307,392	25
BatchNorm2d(192)	192x15x15	384	
MaxPool2d k3, s2	192x7x7	0	33
Conv2d 192->384, k3, s1, p1	384x7x7	663,936	49
BatchNorm2d(384)	384x7x7	768	
Conv2d 384->384, k3, s1, p1	384x7x7	1,327,488	65
BatchNorm2d(384)	384x7x7	768	
Conv2d 384->256, k3, s1, p1	256x7x7	884,992	81
BatchNorm2d(256)	256x7x7	512	
MaxPool2d k3, s2	256x3x3	0	97
Linear 2304->1024	1024	2,360,320	
Linear 1024->1024	1024	1,049,600	
Linear 1024->10	10	10,250	

PyTorch Zoo AlexNet (torchvision, final layer resized): 57,044,810 parameters, first dense layer input = 9,216

11x11 stride-4 first convolution, so a 64x64 input shrinks to 15 -> 7 -> 3 -> 1 spatially; AdaptiveAvgPool2d forces a 6x6 map (256 x 6 x 6 = 9,216 features) into 4096-wide dense layers. This is why it has about 57M parameters but fewer MACs than AlexNet-64 on this input.

Layer	Output (CxHxW or F)	Params	RF (px)
Conv2d 3->64, k11, s4, p2	64x15x15	23,296	11
MaxPool2d k3, s2	64x7x7	0	19
Conv2d 64->192, k5, s1, p2	192x7x7	307,392	51
MaxPool2d k3, s2	192x3x3	0	67
Conv2d 192->384, k3, s1, p1	384x3x3	663,936	99
Conv2d 384->256, k3, s1, p1	256x3x3	884,992	131
Conv2d 256->256, k3, s1, p1	256x3x3	590,080	163
MaxPool2d k3, s2	256x1x1	0	195
AdaptiveAvgPool2d (6, 6)	256x6x6	0	
Linear 9216->4096	4096	37,752,832	
Linear 4096->4096	4096	16,781,312	
Linear 4096->10	10	40,970	

Reference check: the handout quotes nominal sizes of about 0.34M (LeNet), 0.45M (Modernized), 6.31M (AlexNet-64) and 57.04M (Zoo) parameters. The measured values here are 0.338M, 0.446M, 6.61M (6.611M with BatchNorm) and 57.04M. The AlexNet-64 count follows directly from the layer specification in Section 3.2 of the handout (convolutions 3,188,672 + FC 2,360,320 + 1,049,600 + 10,250 = 6,608,842), so the handout's 6.31M is treated as a rounded nominal value.

2. Empirical Results and Figures

Table 7.3a: Architectural configuration and computational complexity (CIFAR-10, 10 classes, input 1x3x64x64)

Model	Conv depth	Normalization / regularization	Data aug.	Params (M)	FLOPs (M)	Mean epoch time (s)	Latency (ms)
1. Base LeNet-5	2	None	None	0.338	7.17	21.4	0.34
2. Modernized LeNet	2	None (ReLU + Overlapping MaxPool)	None	0.446	9.19	21.5	0.36
3. AlexNet (No Norm)	5	Dropout (p=0.5)	None	6.609	436.70	24.0	2.11
4. AlexNet + LRN	5	LRN + Dropout (p=0.5)	None	6.609	436.70	26.6	1.95
5. AlexNet + BatchNorm	5	BatchNorm2d + Dropout (p=0.5)	None	6.611	437.97	25.7	1.57
6. AlexNet + BN + Aug	5	BatchNorm + Dropout (p=0.5)	Pure PyTorch Aug	6.611	437.97	53.9	1.57
7. PyTorch Zoo AlexNet	5	PyTorch Zoo (AdaptiveAvgPool2d)	None	57.045	188.20	25.7	1.94

FLOPs = 2 x MACs counted by the lab profiler (thop). Latency is the median of 50 forward passes at batch size 1 on the device above (compare_models.py). Epoch times are from the CIFAR-10 runs listed in Table 7.3b. The handout lists nominal FLOPs of about 4.8 / 8.2 / 250-257 / 310-315 M; the measured counts differ because the nominal figures are rough estimates (for example the AlexNet-64 convolutions alone are 214.9 M MACs by hand count).

Table 7.3b: Validation performance across datasets (single run, seed 42)

Model	CIFAR-10 epochs	CIFAR-10 Val Acc (%)	CIFAR-10 Val Macro F1 (%)	Tiny ImageNet epochs	Tiny ImageNet Val Acc (%)	Tiny ImageNet Val Macro F1 (%)
1. Base LeNet-5	10	55.23	55.29	5	10.34	8.19
2. Modernized LeNet	10	66.65	66.82	5	19.14	17.29
3. AlexNet (No Norm)	10	73.52	73.48	5	14.87	11.92
4. AlexNet + LRN	10	75.46	75.29	5	14.69	11.55
5. AlexNet + BatchNorm	10	79.93	80.06	5	20.53	18.04
6. AlexNet + BN + Aug	15	82.83	82.90	5	19.66	16.79
7. PyTorch Zoo AlexNet	10	64.90	64.99	n/a	n/a	n/a

Rows 3-5 and 7 use --no-augment so that 'Data Augmentation = None' holds; row 6 uses the augmentation pipeline. Row 6 on CIFAR-10 is the handout's 15-epoch run; its no-augmentation counterpart (15 epochs) is in Table 3. Tiny ImageNet runs are 10 epochs each.

Table 7.3c: Multi-seed evaluation on CIFAR-10 (Mean +/- Std over seeds 42, 43, 44)

Configuration	Val Acc (%)	Val Macro F1 (%)	Final train-val gap (pts)	Mean epoch time (s)
Base LeNet-5 (10 ep, no aug)	55.23 +/- 0.00% (n=1)	55.29 +/- 0.00% (n=1)	13.99 +/- 0.00 (n=1)	21.4 +/- 0.0 (n=1)
Modernized LeNet (10 ep, no aug)	66.65 +/- 0.00% (n=1)	66.82 +/- 0.00% (n=1)	13.99 +/- 0.00 (n=1)	21.5 +/- 0.0 (n=1)
AlexNet No Norm (10 ep, no aug)	73.52 +/- 0.00% (n=1)	73.48 +/- 0.00% (n=1)	1.32 +/- 0.00 (n=1)	24.0 +/- 0.0 (n=1)
AlexNet + LRN (10 ep, no aug)	75.46 +/- 0.00% (n=1)	75.29 +/- 0.00% (n=1)	3.57 +/- 0.00 (n=1)	26.6 +/- 0.0 (n=1)
AlexNet + BatchNorm (10 ep, no aug)	79.93 +/- 0.00% (n=1)	80.06 +/- 0.00% (n=1)	4.93 +/- 0.00 (n=1)	25.7 +/- 0.0 (n=1)
AlexNet + BN (15 ep, no aug)	81.18 +/- 0.00% (n=1)	81.08 +/- 0.00% (n=1)	11.72 +/- 0.00 (n=1)	25.0 +/- 0.0 (n=1)
AlexNet + BN + Aug (15 ep)	82.83 +/- 0.00% (n=1)	82.90 +/- 0.00% (n=1)	2.99 +/- 0.00 (n=1)	53.9 +/- 0.0 (n=1)

Std is the sample standard deviation (ddof = 1, utils/seed.py). With only three seeds these are rough estimates of run-to-run variation.

Table 7.3d: Extended ablation exploration (CIFAR-10, seed 42, Delta vs reference run)

Experiment	Reference	Val Acc (%)	Macro F1 (%)	Epoch time (s)	Delta Acc (pts)	Delta F1 (pts)	Delta time (s)
AlexNet+BN, lr 1e-4	5. AlexNet + BatchNorm	n/a	n/a	n/a	n/a	n/a	n/a

Experiment	Reference	Val Acc (%)	Macro F1 (%)	Epoch time (s)	Delta Acc (pts)	Delta F1 (pts)	Delta time (s)
AlexNet+BN, lr 1e-2	5. AlexNet + BatchNorm	n/a	n/a	n/a	n/a	n/a	n/a
AlexNet+BN, batch 64	5. AlexNet + BatchNorm	n/a	n/a	n/a	n/a	n/a	n/a
AlexNet+BN, batch 256	5. AlexNet + BatchNorm	n/a	n/a	n/a	n/a	n/a	n/a
AlexNet No Norm + Aug	3. AlexNet (No Norm)	n/a	n/a	n/a	n/a	n/a	n/a
Modernized LeNet + Aug	2. Modernized LeNet	n/a	n/a	n/a	n/a	n/a	n/a

Custom experiments chosen to probe (a) learning-rate and batch-size sensitivity of the best architecture and (b) whether augmentation helps a network that has no normalization and a shallow network. References are the 10-epoch no-augmentation runs of Table 7.3b.

Figures

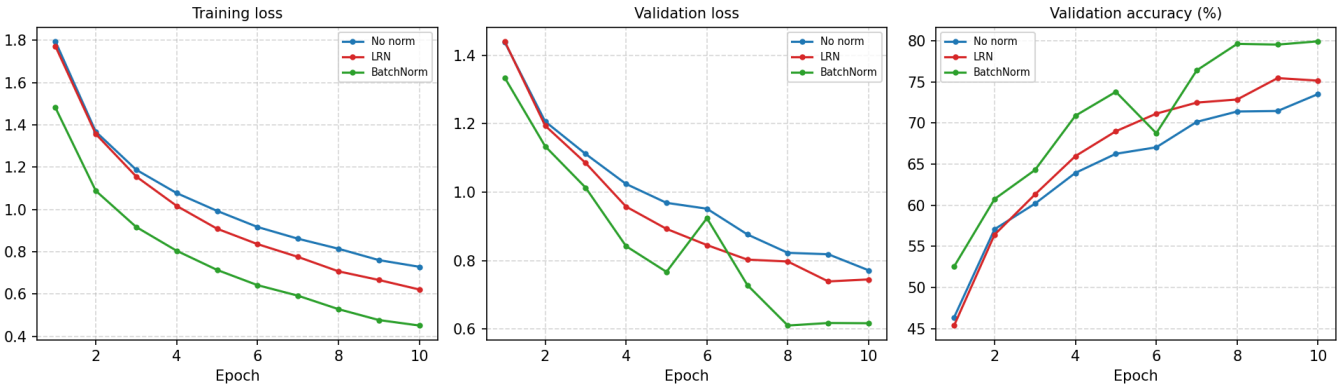


Figure 1: Normalization study, AlexNet-64 on CIFAR-10, 10 epochs (training loss, validation loss, validation accuracy).

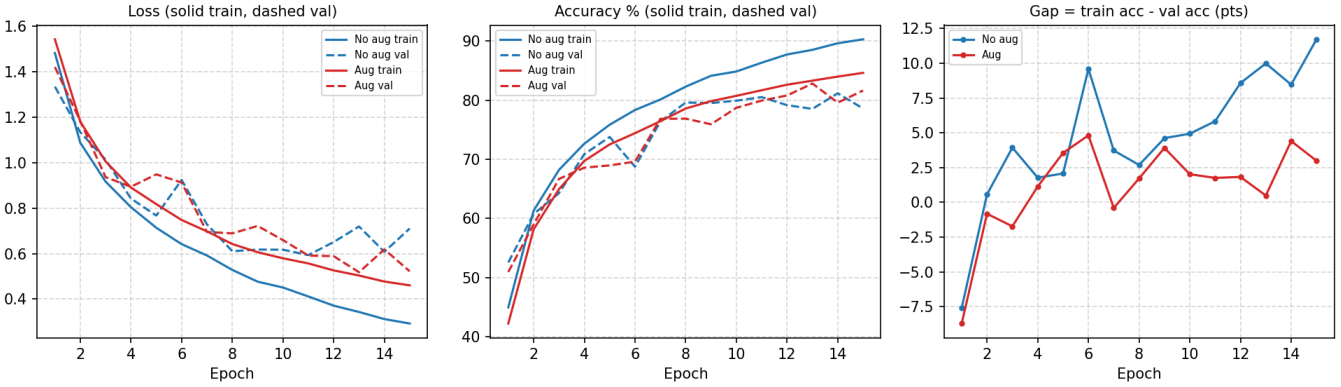


Figure 2: Effect of data augmentation, AlexNet-64 + BatchNorm on CIFAR-10, 15 epochs (loss, accuracy and generalization gap).

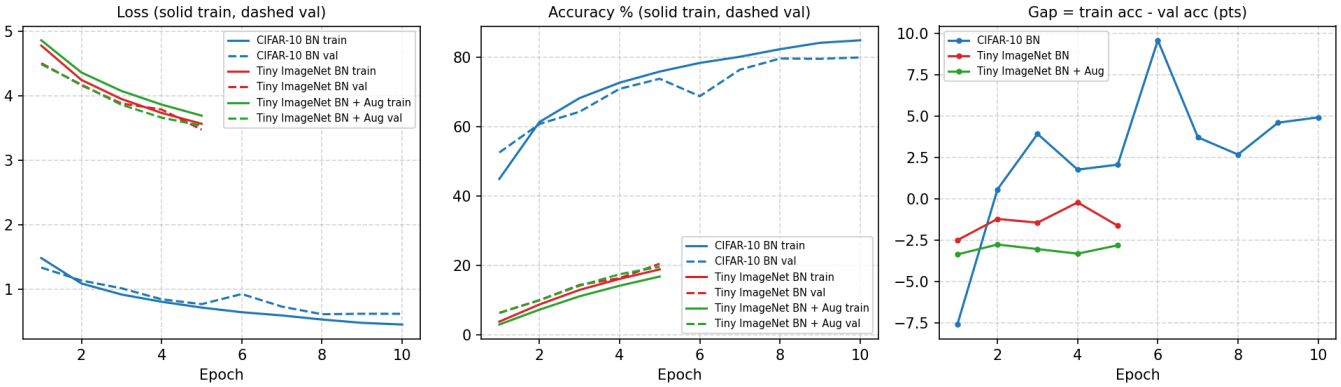


Figure 3: CIFAR-10 versus Tiny ImageNet-200, AlexNet-64 + BatchNorm (loss, accuracy and gap per epoch).

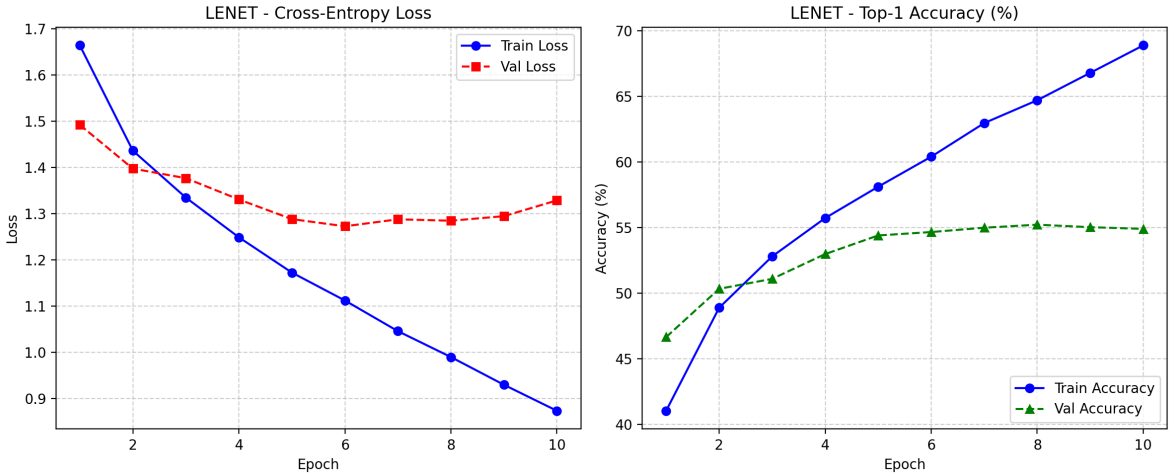


Figure 4: Baseline LeNet-5 loss/accuracy curves (train.py --plot-file).

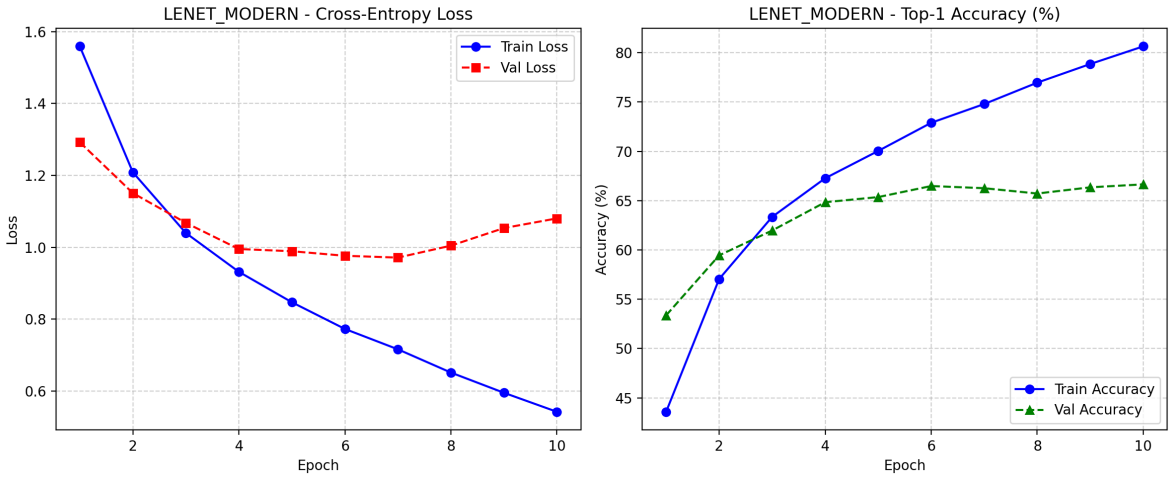


Figure 5: Modernized LeNet loss/accuracy curves.

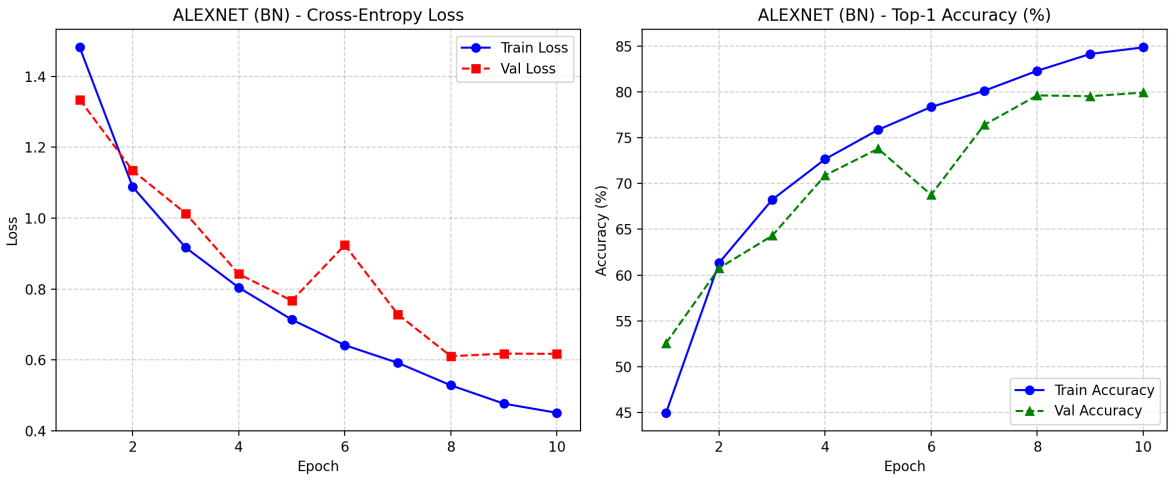


Figure 6: AlexNet-64 + BatchNorm loss/accuracy curves.

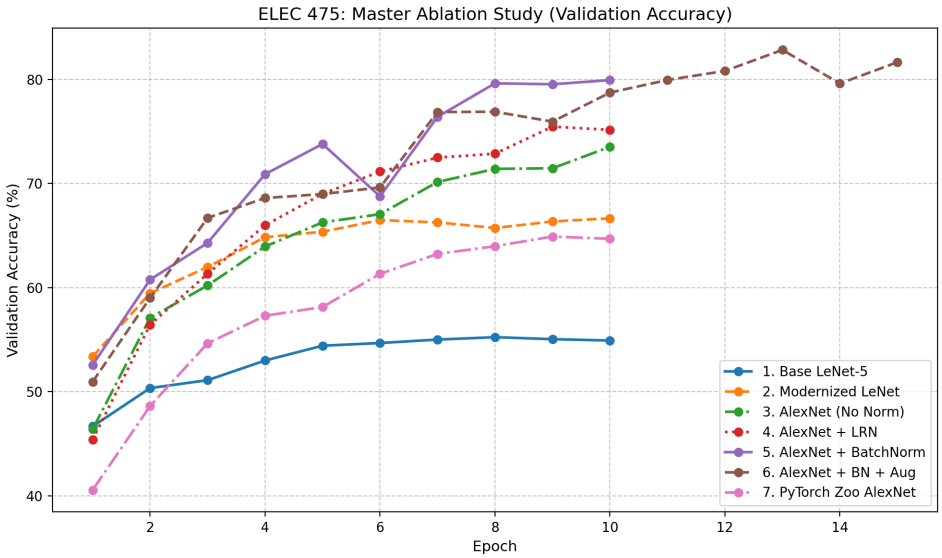


Figure 7: Master ablation summary, validation accuracy of the seven-model ladder on CIFAR-10 (python -m utils.visualization --plot-ablation).

Exploratory data analysis (python -m utils.eda --dataset both)

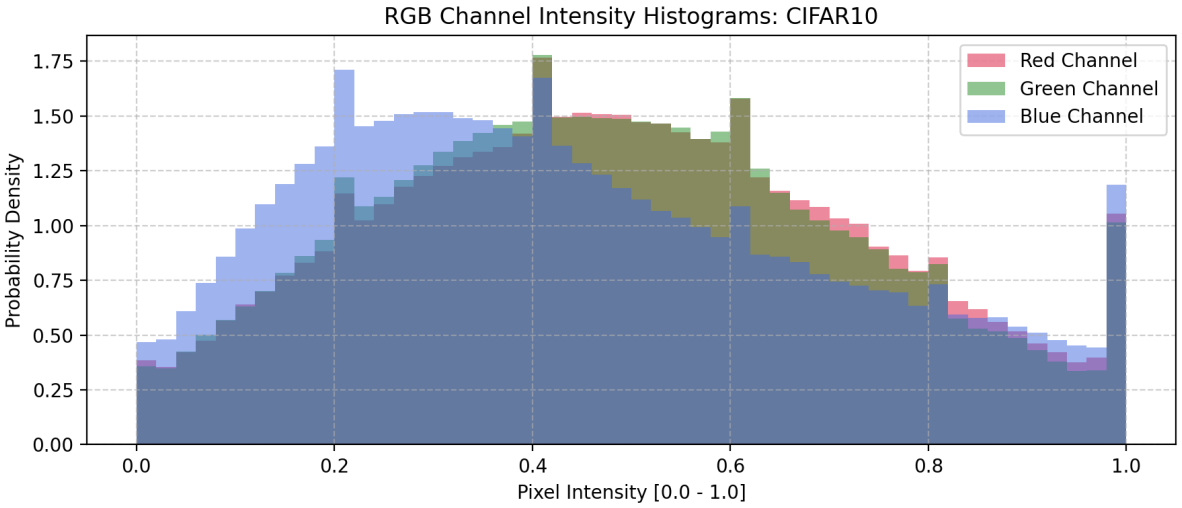


Figure 8: eda_cifar10_channel_histograms.png

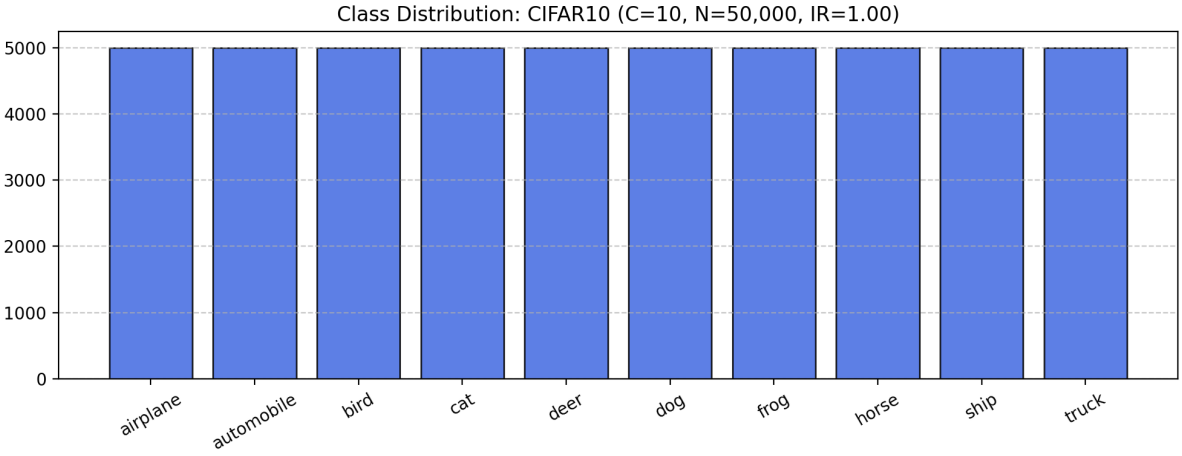


Figure 9: eda_cifar10_class_distribution.png

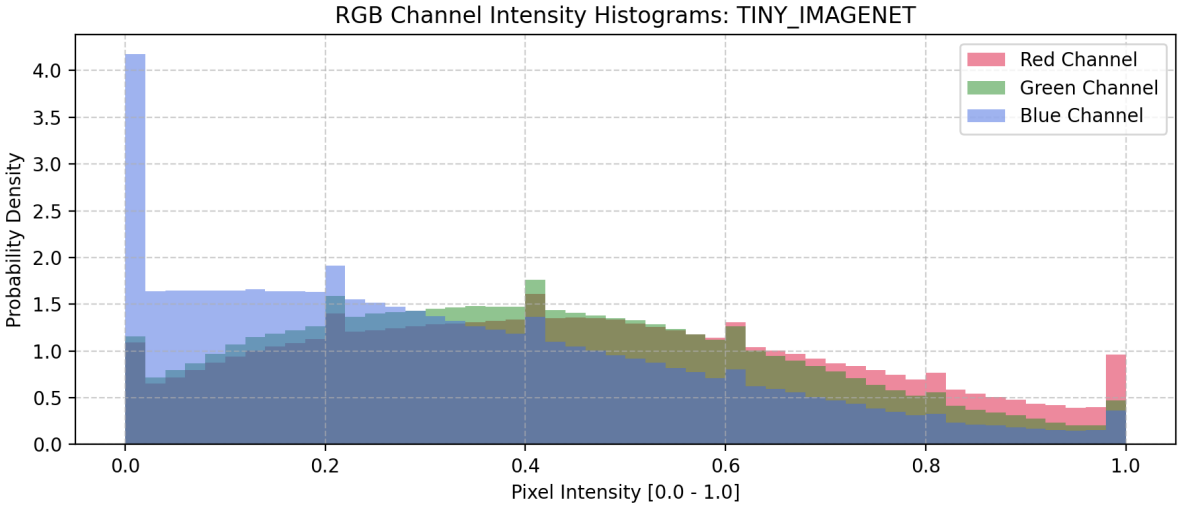


Figure 10: eda_tiny_imagenet_channel_histograms.png

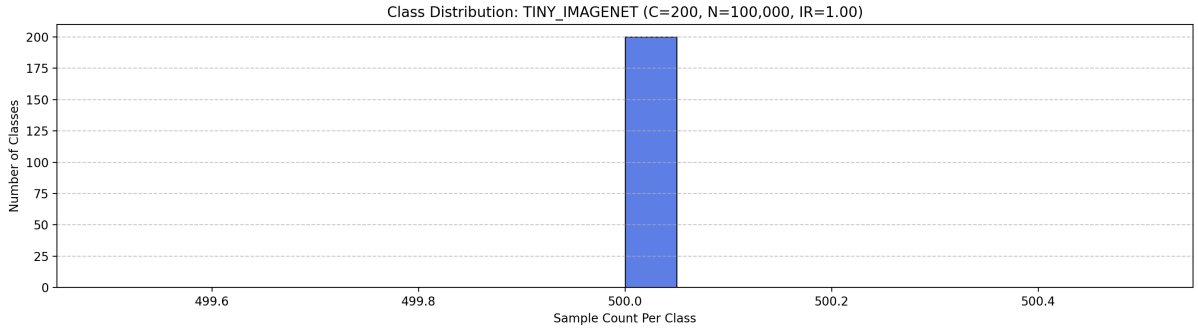


Figure 11: eda_tiny_imagenet_class_distribution.png

Transform paradigm diagnostic (python demo_transforms.py)

```
================================================================================
ELEC 475: Transform Paradigm & Variable Type Diagnostic
================================================================================

[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Width] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cuda:0 (NVIDIA GPU VRAM)

--------------------------------------------------------------------------------
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
--------------------------------------------------------------------------------
- OpenCV (CPU)            :   10.41 ms  ( 96,019 img/s) + Host-to-Device Copy required
- NumPy (CPU)             :   38.19 ms  ( 26,186 img/s) + Host-to-Device Copy required
- PyTorch (GPU Tensors)   :  217.29 ms  (  4,602 img/s) [ZERO PCIe Transfer Overhead]
================================================================================
```

Model complexity benchmark (python compare_models.py --dataset cifar10)

```
================================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device        : cuda (Tesla T4)
Dataset       : CIFAR10 (10 classes)
Input Shape   : (1, 3, 64, 64)
================================================================================

Instantiating models for profiling...
Profiling FLOPs, Multiply-Accumulates, and Latency...

| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms) | Throughput (img/s) |
| ----------------------------|------------|----------|-----------|--------------|------------------- |
| 1. LeNet-5 (Base)           | 0.338      | 3.59     | 7.17      | 0.34         | 2910.5             |
| 2. LeNet-Modern (ReLU+Pool) | 0.446      | 4.59     | 9.19      | 0.36         | 2811.6             |
| 3. AlexNet-64 (None)        | 6.609      | 218.35   | 436.70    | 2.11         | 473.2              |
| 4. AlexNet-64 (LRN)         | 6.609      | 218.35   | 436.70    | 1.95         | 512.7              |
| 5. AlexNet-64 (BatchNorm)   | 6.611      | 218.98   | 437.97    | 1.57         | 635.5              |
| 6. Torchvision Zoo AlexNet  | 57.045     | 94.10    | 188.20    | 1.94         | 515.9              |

================================================================================
Academic Notes on Complexity Metrics:
1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point Operations (FLOPs).
2. Memory vs Compute: Dense Linear layers contribute the majority of parameters, but Conv layers dominate FLOPs.
3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).
================================================================================
```

3. Technical Analysis and Discussion

3.1 Normalization study (None vs LRN vs BatchNorm2d, CIFAR-10, 10 epochs)

Variant	Val Acc (%)	Val Macro F1 (%)	Epoch time (s)	FLOPs (M)	Val-loss up-ticks	Total val-loss rise	Train-loss step std
No normalization	73.52	73.48	24.0	436.70	0	0.000	0.117
LRN	75.46	75.29	26.6	436.70	1	0.006	0.113
BatchNorm2d	79.93	80.06	25.7	437.97	2	0.164	0.107

Performance improvement: Yes. BatchNorm reached 79.93% accuracy and 80.06% Macro F1, versus 73.52% / 73.48% without normalization and 75.46% / 75.29% with LRN. Relative to the better of the other two the difference is +4.47 pts accuracy and +4.77 pts Macro F1.

Why: Batch Normalization standardizes each channel with the mini-batch mean and variance and then re-scales with learnable gamma and beta. This keeps the distribution of every layer's input roughly fixed while earlier layers change, which is the internal covariate shift argument of Ioffe and Szegedy (2015), so each layer no longer has to keep chasing a moving input distribution. Later analysis (Santurkar et al., 2018) attributes much of the benefit to a smoother loss surface: with normalized activations the gradient changes less abruptly between steps, so the gradient direction stays predictive over larger steps. In practice this allows the same learning rate (1e-3) to make faster progress, reduces sensitivity to initialization, and the per-batch noise acts as a mild regularizer.

Computational overhead: Yes. Mean epoch time was 24.0 s (none), 26.6 s (LRN, +10.5%) and 25.7 s (BatchNorm, +7.0%); a change of 5% or more is treated as measurable. Profiler FLOPs were 436.70 M, 436.70 M and 437.97 M. Element-wise normalization is cheap in arithmetic: the five BatchNorm layers touch about 159 thousand activations per image, i.e. a few hundred thousand extra operations against roughly 437 M FLOPs in the convolutions and dense layers (well under 1%). LRN touches 109 thousand activations (stages 1-2) but each needs a sum of squares over 5 channels plus a power, so it costs more per element than BatchNorm. What matters in wall-clock time is memory traffic and kernel launches rather than FLOPs, because these layers are memory-bound (read the whole activation tensor, write it back); training also needs batch statistics and extra backward kernels. Against that small cost, the convergence benefit (when present) is measured in epochs, not percent.

Convergence stability. Counting epochs in which validation loss went up and the summed size of those increases (table above), the most stable trajectory was **No normalization** (0 up-tick(s), total rise 0.000). Validation loss after epoch 1 was 1.438 (none), 1.441 (LRN) and 1.334 (BatchNorm) and the minimum was 0.772, 0.739, 0.610. See Figure 1 for the full curves; a normalization scheme that lowers the first-epoch loss and keeps later increases small provides the steadiest gradient progression.

3.2 Data augmentation and the generalization gap (AlexNet-64 + BatchNorm, CIFAR-10, 15 epochs)

Run	Train Acc (%) (final)	Val Acc (%) (final)	Gap (pts)	Best Val Acc (%)	Best Val Macro F1 (%)	Min val loss
No augmentation	90.31	78.59	11.72	81.18	81.08	0.593
Pure PyTorch augmentation	84.64	81.65	2.99	82.83	82.90	0.518

Generalization gap reduction: Yes. Without augmentation the gap was 90.31 - 78.59 = 11.72 pts; with augmentation 84.64 - 81.65 = 2.99 pts. Gap values are taken at the final epoch. Note that training accuracy is the running average over each epoch, measured in training mode (dropout active) and, for the augmented run, on randomly transformed images, so part of any gap reduction comes from the training images becoming harder rather than from better validation performance; the validation numbers below are the cleaner evidence.

Validation metric improvement: Yes. Best validation accuracy changed by +1.65 pts (81.18% to 82.83%) and Macro F1 by +1.82 pts (81.08% to 82.90%).

Why: random crops (4-pixel reflect padding), horizontal flips and brightness/contrast/saturation jitter are label-preserving, so every epoch shows the network a different version of each image. The network can no longer lower the loss by memorizing exact pixel patterns at fixed positions, so its filters are pushed toward features that stay useful under small translations, mirroring and lighting change (invariant semantic features). This reduces variance and lets the larger model keep improving on held-out data.

3.3 Dual-dataset benchmark (Tiny ImageNet-200)

Generalization above random chance: Yes. AlexNet-64 + BatchNorm with augmentation (the handout's Step 4 command) reached 19.66% validation accuracy and 16.79% Macro F1 on 200 classes, against a random-guess baseline of 0.5% (without augmentation: 20.53% / 18.04%).

Cross-dataset comparison. On CIFAR-10 the same architecture (10 epochs, no augmentation) scored 79.93% accuracy and 80.06% Macro F1, so Tiny ImageNet is 60.3 pts lower in accuracy. Reasons: (i) 200 fine-grained classes instead of 10 coarse ones (chance 0.5% versus 10%), including many visually similar categories; (ii) only 500 training images per class versus 5,000 for CIFAR-10, so far fewer examples to learn each class; (iii) Tiny ImageNet images are natively 64x64 natural photographs with cluttered backgrounds and large intra-class variation, while CIFAR-10 images are 32x32 objects up-sampled to 64x64 and carry less detail; (iv) the 1024-unit dense layers and a 200-way output give the network ample capacity to memorize 100,000 images, so overfitting is a larger concern.

Overfitting trajectory. With augmentation the Tiny ImageNet validation loss reached its minimum 3.535 at epoch 5 of 5 and ended at 3.535, so it was still decreasing or had plateaued without clear divergence; the final train/val accuracy gap was -2.8 pts. Without augmentation the Tiny ImageNet validation loss reached its minimum 3.474 at epoch 5 and ended at 3.474. On CIFAR-10 (10 epochs, no augmentation) validation loss bottomed out at 0.610 in epoch 8 and ended at 0.617 (gap 4.9 pts). See Figure 3. Tiny ImageNet has fewer samples per class and many more classes, so the network can fit the training set faster than it can learn generalizable features, which shows up as an earlier validation-loss minimum and a larger train/validation gap whenever that is observed above.

3.4 Differences between the model variations (ladder summary)

2. Modernized LeNet versus 1. Base LeNet-5: accuracy 55.23% to 66.65% (+11.42 pts), parameters 0.34 M to 0.45 M, MFLOPs 7.2 to 9.2, epoch time 21.4 s to 21.5 s. 3. AlexNet (No Norm) versus 2. Modernized LeNet: accuracy 66.65% to 73.52% (+6.87 pts), parameters 0.45 M to 6.61 M, MFLOPs 9.2 to 436.7, epoch time 21.5 s to 24.0 s. 4. AlexNet + LRN versus 3. AlexNet (No Norm): accuracy 73.52% to 75.46% (+1.94 pts), parameters 6.61 M to 6.61 M, MFLOPs 436.7 to 436.7, epoch time 24.0 s to 26.6 s. 5. AlexNet + BatchNorm versus 4. AlexNet + LRN: accuracy 75.46% to 79.93% (+4.47 pts), parameters 6.61 M to 6.61 M, MFLOPs 436.7 to 438.0, epoch time 26.6 s to 25.7 s. 6. AlexNet + BN + Aug versus 5. AlexNet + BatchNorm: accuracy 79.93% to 82.83% (+2.90 pts), parameters 6.61 M to 6.61 M, MFLOPs 438.0 to 438.0, epoch time 25.7 s to 53.9 s. Zoo AlexNet versus AlexNet-64 + BN: parameters 6.61 M to 57.04 M, MFLOPs 438 to 188, accuracy 79.93% to 64.90%.

Interpretation. (1) Base LeNet to Modernized LeNet: ReLU does not saturate for positive inputs (derivative exactly 1) so gradients are not shrunk through the layers, overlapping max pooling keeps the strongest local response instead of blurring it, and padding 2 keeps border information and feeds a larger feature vector to the dense layers; these changes cost only a small amount of extra computation. (2) Modernized LeNet to AlexNet: five convolution stages with 64 to 384 channels build a much richer feature hierarchy (the receptive field reaches 97 pixels at the last pooling layer, larger than the 64-pixel input, see the RF column) at roughly 50 to 60 times the FLOPs of LeNet, with dropout controlling the capacity of the dense layers. (3) Normalization changes training dynamics and, per Section 3.1, adds very little arithmetic. (4) Augmentation does not change parameters, FLOPs or inference latency at all; it only changes the training distribution, so its benefit is free at inference time but costs training-time data processing. (5) The PyTorch Zoo AlexNet has about 8.6 times the parameters of AlexNet-64 (its dense layers see 9,216 features and are 4096 wide) but fewer MACs here because its 11x11 stride-4 first layer shrinks the 64x64 input immediately; more parameters do not translate into a proportional accuracy gain at this input size and training length.

Limitations. Most results are single-seed (see Table 7.3c for variation where extra seeds were run); runs are short (10 to 15 epochs); the CIFAR-10 test split doubles as the validation set (no separate hold-out), so best-epoch selection is slightly optimistic; epoch time includes the validation pass and data loading with two workers, so for small networks it measures the data pipeline as much as the model; and absolute timings depend on the GPU and CPU used.

4. Reproducibility

All commands are listed in commands.txt. In short: `pip install -r requirements.txt`, `python run_all.py`, `python make_report.py --name ... --student-number ...`. Global seeding (Python, NumPy, PyTorch, cuDNN deterministic) is in utils/seed.py. The data/ folder is not part of the submission; datasets download automatically on first use. Deviations from the handout text: rows 3-5 add --no-augment (augmentation defaults on); LRN follows Figure 4.2 (after ReLU, stages 1-2 only); the LeNet baselines are trained for 10 epochs (in-lab used 5) so that all rows share a comparable training

length.